SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE:CSA1741

CO1-AT1-Analytical Problem Solving

Register Number	**192312070**
Name	**Balaguru S**

COURSE OUTCOME COVERED

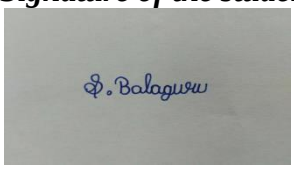
CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weight age: 40%

Code of Conduct:

I Balaguru S (192312070) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem	10
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method	10
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process	12.5
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations	10
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation	7.5

1. Solve the Water Jug problem: a 4-gallon jug and a 3-gallon jug, no markings, and a pump. How can you get exactly 2 gallons of water into the 4-gallon jug?

Answer

Problem Understanding

- Jugs: X = 4-gallon jug, Y = 3-gallon jug. Neither has markings.
- Allowed operations: fill a jug from the pump, empty a jug on the ground, pour from one jug into the other.
- Goal: exactly 2 gallons in the 4-gallon jug, i.e. state (2, n) for any n.

Method Selection

The problem is formulated as a state-space search problem and solved with Breadth-First Search (an uninformed strategy), which guarantees the shortest sequence of moves.

- State: (x, y) where x = water in the 4-gallon jug (0–4) and y = water in the 3-gallon jug (0–3).
- Initial state: (0, 0)
- Goal state: (2, y)

Production rules (actions)

No.	Rule	Description
1	(x, y) if $x < 4 \rightarrow (4, y)$	Fill the 4-gallon jug
2	(x, y) if $y < 3 \rightarrow (x, 3)$	Fill the 3-gallon jug
3	(x, y) if $x > 0 \rightarrow (0, y)$	Empty the 4-gallon jug on the ground
4	(x, y) if $y > 0 \rightarrow (x, 0)$	Empty the 3-gallon jug on the ground
5	(x, y) if $x + y \geq 4$ and $y > 0 \rightarrow (4, y - (4 - x))$	Pour from 3-gallon jug into 4-gallon jug until it is full
6	(x, y) if $x + y \geq 3$ and $x > 0 \rightarrow (x - (3 - y), 3)$	Pour from 4-gallon jug into 3-gallon jug until it is full
7	(x, y) if $x + y \leq 4$ and $y > 0 \rightarrow (x + y, 0)$	Pour all water from 3-gallon jug into 4-gallon jug
8	(x, y) if $x + y \leq 3$ and $x > 0 \rightarrow (0, x + y)$	Pour all water from 4-gallon jug into 3-gallon jug

Solution Process and Calculation

Step	4-gal jug (x)	3-gal jug (y)	Action (rule applied)
0	0	0	Initial state
1	4	0	Fill the 4-gallon jug (Rule 1)
2	1	3	Pour from 4-gal into 3-gal until full: $4 - 3 = 1$ left (Rule 6)
3	1	0	Empty the 3-gallon jug (Rule 4)

Step	4-gal jug (x)	3-gal jug (y)	Action (rule applied)
4	0	1	Pour the 1 gallon from 4-gal into 3-gal (Rule 8)
5	4	1	Fill the 4-gallon jug again (Rule 1)
6	**2**	3	Pour from 4-gal into 3-gal until full: the 3-gal jug needs 3 − 1 = 2, so 4 − 2 = **2** remain (Rule 6)

Alternative solution (also 6 steps): (0,0) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0).

Interpretation of Results

The goal state (2, 3) is reached in 6 moves, so exactly 2 gallons are in the 4-gallon jug. This works because the difference between the two jug capacities (4 − 3 = 1) can be accumulated, and 4 − 2 × 1 = 2. BFS shows that no sequence shorter than 6 moves reaches the goal.

2. Mars Rover – analyse and explore samples on Mars. (a) What are the percepts for this agent? (b) Characterize the operating environment. (c) What are the actions the agent can take? (d) How can one evaluate the performance of the agent? (e) What sort of agent architecture is most suitable, and why?

Answer

Problem Understanding

The Mars Rover is an autonomous intelligent agent that must move across unknown Martian terrain, collect and analyse rock and soil samples, and send results to Earth. It can be described using the PEAS framework (Performance measure, Environment, Actuators, Sensors).

Method Selection

PEAS description, task-environment properties, and the standard agent types (simple reflex, model-based, goal-based, utility-based, learning) are used to select the architecture.

(a) Percepts (inputs from sensors)

- Camera images (stereo/panoramic) of terrain, rocks and obstacles
- Spectrometer / chemical analyser readings of rock and soil samples
- Wheel encoders, odometer, gyroscope and inclinometer (position, speed, tilt)
- Terrain sensors: slope, wheel slip, obstacle/proximity (LIDAR) data
- Weather sensors: temperature, wind, dust and radiation levels
- Internal status: battery/power level, solar panel output, instrument health
- Commands and signals received from Earth/orbiter via the antenna

(b) Operating environment (task-environment properties)

Property	Characterization	Reason
Observability	Partially observable	Sensors see only the nearby area; hidden rocks, craters and soil conditions are unknown
Agents	Single agent	The rover acts alone (other rovers/orbiters do not compete)

Property	Characterization	Reason
Determinism	Stochastic	Wheel slip, dust storms and sensor noise make outcomes uncertain
Episodic / Sequential	Sequential	Current moves affect battery, position and later tasks
Static / Dynamic	Dynamic	Weather, light and temperature change while the rover decides
Discrete / Continuous	Continuous	Positions, speeds, temperatures and time are continuous
Known / Unknown	Largely unknown	Terrain and sample content are not known in advance

(c) Actions (actuators)

- Drive forward/backward, steer, turn, stop (wheels and motors)
- Avoid or go around obstacles and climb slopes safely
- Deploy and move the robotic arm; drill, scoop or scrape soil and rock
- Place a sample into an analysis instrument and run it
- Take photographs; point camera and antenna
- Store samples; transmit data to Earth/orbiter
- Go into sleep/safe mode and recharge using solar panels

(d) Performance measure

- Number and scientific value (variety, quality) of samples analysed
- Area explored and number of useful sites reached
- Safety: no damage, no getting stuck, no falls or collisions
- Energy efficiency and use of battery power
- Accuracy and completeness of data transmitted to Earth
- Mission lifetime and time taken to complete objectives

(e) Most suitable agent architecture

A **goal-based agent with utility-based decision making and a model of the world** (a hybrid, layered architecture), with a learning component.

- *Model-based:* the environment is partially observable, so the rover must keep an internal map and state (position, battery, sample list).
- *Goal-based:* it has explicit goals (reach a site, collect a sample, analyse it, transmit), so it must plan sequences of actions (search/planning).
- *Utility-based:* it must trade off conflicting objectives such as science value versus risk and battery use, which a utility function handles.
- *Reactive layer:* simple reflex rules (stop if tilt is too high, avoid obstacle) give fast, safe responses.
- *Learning:* improves terrain classification and navigation from experience.
- Autonomy is essential because the one-way signal delay to Earth (roughly 4–24 minutes) prevents real-time remote control.

Interpretation of Results

Because the rover works in a partially observable, stochastic, dynamic and unknown world, a plain reflex agent is not enough. A layered goal/utility-based, model-based agent can plan, balance risk against reward and still react quickly to danger, which maximizes the performance measure.

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem: place 8 queens on a chessboard such that no queen attacks any other. The configuration in the figure is not a solution because the queen in the first column is on the same diagonal as the queen in the last column.

Answer

Problem Understanding

Place 8 queens on an 8 × 8 board so that no two queens share the same row, column or diagonal. In the given figure the first-column and last-column queens lie on the same diagonal, so the goal test fails and the configuration is not a solution.

Method Selection

The problem is formulated as a state-space search problem and explored with uninformed strategies: Breadth-First Search (BFS) and Depth-First Search (DFS) with backtracking. DFS is preferred because every solution lies at the same depth (8) and the path itself is irrelevant.

Problem formulation (components)

Incremental formulation (basic):

- **States:** any arrangement of 0 to 8 queens on the board.
- **Initial state:** empty board (no queens).
- **Actions:** add a queen to any empty square.
- **Transition model:** returns the board with the queen added to the chosen square.
- **Goal test:** 8 queens are on the board and none is attacked.
- **Path cost:** not important (only the final arrangement matters). Each step may be given cost 1.

Improved formulation (smaller search space):

- **States:** arrangements of n queens ($0 \le n \le 8$), one per column in the leftmost n columns, with no queen attacking another.
- **Actions:** add a queen to the leftmost empty column in a row that is not attacked by any existing queen.

Solution Process and Calculation

- Basic formulation: $64 \times 63 \times 62 \times \ldots \times 57 \approx 1.8 \times 10^{14}$ possible sequences.
- Improved formulation: only 2,057 states, because attacked squares are never considered.
- **DFS with backtracking:** place a queen in column 1, then column 2 in a non-attacked row, and so on. If a column has no safe row, backtrack to the previous column and try the next row.

One valid solution found by this search (row number of the queen in columns 1 to 8):

Column	1	2	3	4	5	6	7	8
Row	1	5	8	6	3	7	2	4

Verification: all rows are different, every column has one queen, and the values of (column − row) are 0, −3, −5, −2, 2, −1, 5, 4 and (column + row) are 2, 7, 11, 10, 8, 13, 9, 12, all distinct, so no two queens share a diagonal.

Interpretation of Results

The goal test (no two queens attacking) is satisfied by the configuration above, unlike the figure given in the question. Formulating the problem so that illegal placements are never generated reduces the search space from about 1.8×10^{14} to 2,057 states, which makes uninformed search practical. The 8-Queens problem has 92 solutions in total (12 distinct if rotations and reflections are treated as the same).

4. A customer wants to travel from one location to another using the OLA cab booking app and can select a cab type (mini, micro, sedan, shared, prime, etc.). Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

Answer

Problem Understanding

- Input: pickup location, drop location, and the customer's comfort/cab-type preference.
- Output: a booked cab and a route that takes the customer from the pickup point to the destination.
- The environment is fully observable (map, cab availability and fares are known to the app), deterministic and sequential, which suits a problem-solving agent.

Method Selection

A **goal-based problem-solving agent** is used. Its goal is "customer is at the destination". A **utility-based** element is added to choose among cab types, because several options reach the goal, differing in cost, comfort and waiting time. Route finding uses a search algorithm (Uniform Cost Search / A*) over the road network.

Problem formulation

- **Initial state:** customer at the pickup location.
- **Goal state:** customer at the drop location.
- **Actions:** select cab type, book the cab, travel along a road segment from one junction to the next.
- **Path cost:** fare, distance or travel time.
- **Goal test:** current location = destination.

Solution Process – Pseudocode

```
import heapq


# ---------- Problem formulation ----------

class CabProblem:

    def __init__(self, pickup, drop, road_map):

        self.initial = pickup              # initial state

        self.goal = drop                   # goal state

        self.road_map = road_map           # {location: [(next_location, distance_km), ...]}
```

```
    def goal_test(self, state):
        return state == self.goal

    def actions(self, state):
        return self.road_map.get(state, [])   # possible road segments


# ---------- Utility-based choice of cab type ----------
def select_cab(cabs, preference):
    """cabs: list of dicts {type, fare, wait, comfort, available}"""
    best, best_utility = None, float("-inf")
    for cab in cabs:
        if not cab["available"]:
            continue
        # higher comfort is good; higher fare and waiting time are bad
        u = (preference["comfort"] * cab["comfort"]
             - preference["price"] * cab["fare"]
             - preference["time"] * cab["wait"])
        if u > best_utility:
            best, best_utility = cab, u
    return best


# ---------- Search: Uniform Cost Search for the route ----------
def search(problem, cab):
    frontier = [(0, problem.initial, [problem.initial])]   # (cost, state, path)
    explored = set()
    while frontier:
        cost, state, path = heapq.heappop(frontier)        # lowest cost first
        if problem.goal_test(state):
            return path, cost                              # solution
```

```
        if state in explored:
            continue
        explored.add(state)
        for next_state, distance in problem.actions(state):
            if next_state not in explored:
                step_cost = distance * cab["rate_per_km"]
                heapq.heappush(frontier, (cost + step_cost, next_state, path + [next_state]))
    return None, None                                   # failure


# ---------- Goal-based problem-solving agent ----------
def ola_agent(pickup, drop, road_map, cabs, preference):
    # 1. Formulate goal and problem
    problem = CabProblem(pickup, drop, road_map)

    # 2. Choose cab type (utility)
    cab = select_cab(cabs, preference)
    if cab is None:
        return "No cab available"

    # 3. Search for the best route
    route, fare = search(problem, cab)
    if route is None:
        return "No route found"

    # 4. Execute the solution
    print("Booked:", cab["type"])
    for location in route:
        print("Travelling to", location)
    print("Reached destination. Estimated fare:", fare)
    return route, fare
```

The agent formulates the goal (reach the destination), formulates the problem, searches for the best route, and then executes the actions one by one. The utility function lets the same agent recommend a different cab (e.g. Micro for the cheapest fare, Prime for comfort, Shared for the lowest cost) depending on the customer's preference, so the solution is both correct and customer-oriented.

5. A logistics company wants the least-cost path from warehouse S to warehouse G using Uniform Cost Search (UCS) on the weighted delivery network graph.

Answer

Problem Understanding

Find the minimum-cost path from the start node S to the goal node G in the given weighted, directed graph. Edge costs (fuel + time) read from the figure:

Edge	Cost
S → A	1
S → G	12
A → B	3
A → C	1
B → D	3
C → D	1
C → G	2
D → G	3

Method Selection

Uniform Cost Search is used. It always expands the node with the lowest path cost g(n) from a priority queue, so the first time the goal is removed from the queue the path is guaranteed to be optimal (all edge costs are positive).

Solution Process and Calculation

Starting with frontier = {S(0)}:

Step	Node expanded (cost g)	Nodes generated with cost	Frontier after expansion (ordered by cost)
1	S (0)	A = 0+1 = 1, G = 0+12 = 12	A(1), G(12)
2	A (1)	B = 1+3 = 4, C = 1+1 = 2	C(2), B(4), G(12)
3	C (2)	D = 2+1 = 3, G = 2+2 = 4 (better than 12, so G is updated)	D(3), B(4), G(4)
4	D (3)	G = 3+3 = 6 (not better than 4, discarded)	B(4), G(4)
5	B (4)	D = 4+3 = 7 (not better than 3, discarded)	G(4)

Step	Node expanded (cost g)	Nodes generated with cost	Frontier after expansion (ordered by cost)
6	G (4)	Goal test passed: stop	–

Order of expansion: S → A → C → D → B → G

Cost of all possible paths from S to G (for verification)

Path	Total cost
S → A → C → G	1 + 1 + 2 = **4**
S → A → C → D → G	1 + 1 + 1 + 3 = 6
S → A → B → D → G	1 + 3 + 3 + 3 = 10
S → G	12

Final answer: the least-cost path is **S → A → C → G with a total cost of 4** units.

Interpretation of Results

The direct route S → G looks shortest in number of edges but costs 12, three times more than the optimal route. UCS correctly ignores it because it expands nodes in order of cumulative cost and not in order of number of edges. For the company, sending packages through warehouses A and C saves 8 cost units (12 − 4) per delivery. UCS is complete and optimal here because every edge cost is positive.